BookLeaf Author Support & Communication Portal

Technical Assignment 1 · Approach write-up · Daksh Singh · Full-Stack Developer

Problem understanding

Authors need to see their books and raise support tickets about royalties, ISBN and metadata, printing, distribution and production status. Staff need a triage queue, an audit trail and AI help that is useful but never acts on its own. The brief asks for real AI integration: classification, priority scoring and a reply drafted from the company's policies, with a human deciding what is sent.

What I prioritised

A complete, working end-to-end flow first: author registration and login, My Books with exact figures (including zero-royalty and in-production books), tickets with attachments and history, and an admin queue with search, filters, assignment, internal notes, status changes and audit history. Then the AI pipeline, then security and tests. I deliberately left out password reset, e-mail notifications and an off-host backup, and list them as limitations.

Architecture decisions

React, Vite and TypeScript front end; Express API; PostgreSQL with Prisma; Zod schemas in one shared contracts package so web, API and AI code agree on every payload. The AI code is a separate package behind a small interface, so the provider can be swapped or mocked in tests. Server-sent events push admin replies to the author without a manual refresh, with polling as a fallback. Deployed with Docker Compose on a VPS behind Traefik with HTTPS; the database is not exposed publicly.

AI integration approach

Real OpenRouter calls (model nvidia/nemotron-3-super-120b-a12b:free, so cost is zero). A ticket is classified into the six categories and given a priority; the relevant policy passages are retrieved (80/20 royalty split, quarterly payments within 45 days, the Rs 1,000 threshold, ISBN escalation, 5-7 day reprints, 7-10 day listings, 3-5 day metadata updates) and a draft is written from them. Model output is treated as untrusted: a validator rejects invented figures, dates or time frames, links, refunds and claims that an action was already done, and a labelled template draft is shown instead. Analysis never sends anything; the admin edits the draft and explicitly sends it. Timeouts, bad keys, invalid output and rate limits are recorded as attempt states and ticket handling continues manually.

Engineering trade-offs

A free model costs nothing but is non-deterministic and rate-limited (about 25 analyses a day), so I chose strict validation over fluent drafts and some drafts fall back to templates. A few deterministic rules (for example legal or fraud wording, ISBN errors) override the model for predictability, at the cost of some rigidity. A single API instance keeps sessions, rate limits and live updates simple but does not scale out as built. Attachments live on local disk.

Security and reliability

Server-side sessions in HttpOnly cookies, CSRF tokens and origin checks, Argon2id password hashing, role checks, per-author scoping (another author's ticket answers 404 exactly like a missing one), rate limits, strict input validation, forced-download attachments with type checks, and the API key only on the server. Daily encrypted database and attachment backups with an isolated restore test; the previous image is kept for rollback. Verified: 509 AI-package, 292 API, 68 front-end and 41 browser end-to-end tests pass, and typecheck and build are clean.

Current status and honest limits

The live site holds the assignment's 10 authors and 18 books, checked field by field against the original JSON (358 comparisons, 0 mismatches). Real AI was verified locally on the final code: classification and priority were correct for all six categories, and about five of six model drafts passed validation. On the live site the free daily quota was exhausted, so live analyses currently end with a visible rate-limit notice and replies are written manually; I have not demonstrated a successful live inference.

Evolving it into production

Use a funded model with a fallback model and an evaluation set to track classification and draft quality; move sessions, rate limits and events to shared storage so several instances can run; store attachments in object storage and ship encrypted backups off-host; add password reset, e-mail notifications, readiness checks and alerting; add an admin screen to link books to new authors; and add semantic retrieval over a larger knowledge base.